

SIMATS ENGINEERING

ASSESSMENT TOOL 3 – MOCK INTERVIEW

COMPUTER VISION | ITA0515 | SLOT B

1. Design and explain a complete OpenCV pipeline to read, process, and display multiple images from a folder efficiently.

Answer: I would first get the list of image files from the folder and process them one by one. I would check whether each image is loaded correctly before doing any processing. Then I would resize the image if it is very large, convert it to the required color format, apply the required processing operation, and display or save the result.

A simple pipeline would be: folder input → image reading → validation → resizing → preprocessing → required computer-vision operation → display or save output.

In OpenCV, `cv2.imread()` can be used to read an image, `cv2.resize()` to change its size, `cv2.cvtColor()` to change the color space, and `cv2.imshow()` to display it. If many images are processed, I would avoid keeping all full-size images in memory at the same time. I would also close display windows properly after processing.

The main challenges are corrupted files, different image sizes, large files and unsupported formats. Checking the image after reading and processing one file at a time helps handle these issues efficiently.

2. A video file is not loading properly in OpenCV. Analyze the possible causes and propose a structured debugging approach.

Answer: I would debug the problem step by step instead of changing many things at once.

- First, I would check whether the video file path is correct and whether the file actually exists.
- Next, I would open the video using `cv2.VideoCapture()` and check `cap.isOpened()`. If it returns false, the source has not opened correctly.
- I would then check whether the file format and codec are supported by the OpenCV installation.
- I would try another known-working video file. If the second file works, the problem is probably with the original file.
- I would also check whether frames are being returned by testing the return value from `cap.read()`.
- If the video opens but frames are empty, I would check the codec, file integrity and OpenCV/FFmpeg support.
- Finally, I would release the capture object using `cap.release()` and close any windows using `cv2.destroyAllWindows()`.

A useful debugging order is: file path → VideoCapture creation → `isOpened()` → frame reading → codec/support → video integrity → cleanup. This makes it easier to identify the exact stage where the problem occurs.

3. You need to overlay text and shapes on a live video feed. Design your solution and explain how you would implement it efficiently.

Answer: I would open the camera with `cv2.VideoCapture()`, read each frame inside a loop, draw the required shapes and text on that frame, and then display the modified frame.

For example, `cv2.rectangle()` can draw a rectangle, `cv2.circle()` can draw a circle, `cv2.line()` can draw a line, and `cv2.putText()` can add text. These functions modify the current frame before it is displayed.

The basic order is: capture frame → check frame → draw shapes → add text → display frame → check for exit key → continue.

For efficiency, I would avoid unnecessary image copies and keep the frame resolution at a reasonable size. If the overlay depends on an object-detection result, I would calculate the result only as often as needed rather than repeating expensive work unnecessarily.

At the end, I would release the camera and close the display window.

4. An image appears too dark when read. Analyze the issue and propose modifications to your processing pipeline.

Answer: First, I would check whether the image is actually dark or whether the display or processing step is causing the problem. I would verify that `cv2.imread()` has loaded the image correctly and inspect the image before applying other operations.

If the image is genuinely underexposed, I would improve its brightness and contrast. A simple approach is to convert the image to a suitable color space and adjust the intensity component. For a grayscale image, histogram equalization can improve the distribution of intensity values. CLAHE can be used when different parts of the image have different brightness levels.

I would also check the order of operations. For example, excessive thresholding or an incorrect color conversion can make an image appear darker. Therefore, I would inspect the result after each major processing step.

A suitable pipeline is: read image → verify image → inspect original → convert color space if needed → improve brightness/contrast → apply local contrast enhancement when required → display the improved image.

This approach improves visibility while avoiding unnecessary processing and makes it easier to find the actual reason for the dark appearance.